

Runtime-Resident Knowledge for Robot Learning on Apple Silicon

A structured architectural meta-analysis of runtime-resident knowledge for NumiLab

Numan Thabit

NumiLab, Tromsø, Norway

8 August 2026

Core architecture

NumiLab compiles shared prior knowledge once into GPU-addressable tables and compact tensors. Each simulated robot receives a small `ActiveContext` containing the knowledge relevant to its current state. The control loop reads this resident context directly, without document retrieval or database queries.

Abstract

Large-scale robot learning depends on physics throughput, neural-network execution, memory placement, data movement, and scheduling. This paper develops a runtime-resident knowledge architecture for NumiLab in which thousands of simulated robots share one compiled knowledge bank while retaining only their own changing state, recent events, and compact `ActiveContext`. The synthesis combines GPU-native robot simulators, explicit-memory neural systems, Apple Metal, MLX, and NumiLab's documented Metal execution path. Five architectural findings dominate the literature and platform evidence. First, physics, observations, policy inference, and rollout state remain on the accelerator. Second, Apple unified memory removes explicit device-copy boundaries while memory bandwidth, cache behavior, and synchronization remain central to performance. Third, exact knowledge belongs in indexed tables and graphs, learned semantic knowledge belongs in compact tensors, and recent experience belongs in bounded per-agent memory. Fourth, the policy consumes a small `ActiveContext` instead of scanning the complete knowledge bank. Fifth, Metal storage modes, heaps, argument tables, residency sets, indirect execution, and machine-learning encoders provide the mechanisms for a persistent GPU-native implementation. The resulting NumiLab architecture stores one shared knowledge copy, updates contexts when meaningful state changes occur, and keeps the complete control path on the Apple GPU timeline.

Keywords: Apple Silicon; Metal; robotics simulation; reinforcement learning; GPU residency; unified memory; neural policy; explicit memory; event-driven control; NumiLab.

Plain-language summary

NumiLab loads shared robot knowledge once. Each robot stores only changing state, recent events, and a compact `ActiveContext`. Meaningful events update that context directly on the GPU. The policy reads it without text queries, vector databases, document ranking, or CPU round trips.

1. Introduction

A robot control loop repeatedly reads sensors or simulator state, calculates an action, applies that action, advances physics, and records the result. The neural network is the policy that calculates the action. The runtime is the larger system that decides when the policy runs, where its inputs live, how outputs reach the actuators or simulator, and when memory is updated. This distinction is central to performance: a fast policy can still be placed inside a slow execution path.

The practical NumiLab requirement is that many parallel robots begin with shared prior knowledge. Examples include body topology, joint limits, material properties, vulnerable body regions, basic contact semantics, and skill priors. Each robot also needs its own recent history, such as a slip that started 40 ms ago or a failed recovery attempt. The question is not whether these data is stored. The question is how they is accessed with predictable cost while physics and policy inference remain on the Apple GPU.

Retrieval-Augmented Generation (RAG) retrieves documents or passages for a language generator [22]. Robot control uses a different unit of knowledge: exact numeric relations, compact learned features, and recent state. NumiLab therefore represents control knowledge as compiled runtime data. Exact relations use stable IDs and direct indexing; semantic similarity uses compact tensors; recent experience uses fixed per-environment memory. The critical design variables are representation, placement, access pattern, synchronization, and update schedule.

The NumiLab runtime-resident knowledge layer contains one shared read-mostly knowledge bank, a small ActiveContext per environment, and bounded episodic memory per environment. Exact knowledge resolves through stable IDs and direct indexing. Learned similarity operates on compact candidate sets or tensor operations. State changes and events update only the affected contexts. The CPU handles compilation, scheduling, logging, and publication boundaries; the GPU owns the time-critical execution sequence.

The contribution is a concrete runtime architecture derived from GPU robot-learning systems, Apple's Metal execution model, and NumiLab's persistent Metal world graph. It specifies the representations, memory placement, execution order, and data flow required to make shared robot knowledge part of the live control runtime.

2. Terms and conceptual model

Table 1 defines the terms used throughout the paper. Each definition is operational: every term maps to a concrete data structure or execution boundary.

Term	Meaning in this paper
Runtime	The complete running system: physics, sensors, policy, memory, scheduling, logging, and state publication.
Policy	The function—usually a neural network—that maps observations and context to an action.
Runtime-resident	Allocated and addressable for the duration of a run. Access speed is determined by storage placement, locality, and the execution path.

Term	Meaning in this paper
GPU-resident	Available to GPU commands without a CPU copy or CPU-side lookup in the control-step path.
Shared prior knowledge	Read-mostly information that is common to many environments, such as body graphs, materials, and hazards.
ActiveContext	A small, contiguous per-environment record containing only the knowledge currently needed by the policy.
Episodic memory	A bounded per-environment record of recent typed events and outcomes.
Event	A discrete change with control relevance, such as contact-on, contact-off, slip-start, impact, or recovery-trigger.

Table 1. Operational definitions used in the paper.

2.1 Four forms of knowledge

Robot knowledge has different semantics and therefore uses different representations. Exact information uses explicit tables and graphs. Learned information uses vectors or neural parameters. Per-step state uses compact dynamic buffers. Recent experience uses a bounded temporal record. This separation keeps access predictable and preserves the meaning of each knowledge class.

Class	Examples	Preferred representation
Parametric	Policy weights and learned internal features	Fast inference; difficult to inspect or update independently
Shared exact	Body topology, limits, materials, hazard weights, object IDs	Packed arrays, tables, and graphs with stable integer IDs
Shared semantic	Skill prototypes, learned state classes, similarity features	Compact FP16/BF16 or quantized tensors; bounded candidate operations
Per-environment	Current ActiveContext and recent event history	Small fixed-capacity records updated during execution

Table 2. Knowledge classes is represented according to their semantics.

Figure 1. A document-retrieval pipeline performs variable work that is useful for language tasks. The defined runtime path uses fixed-shape state, stable IDs, and a bounded ActiveContext.

2.2 What the policy receives

The policy receives the normal observation plus a small context record rather than the complete knowledge bank. For a humanoid, that record contains the IDs and features of active body regions, current hazards, relevant skills, and a compact summary of recent events. The context has a fixed compiled shape, giving policy inference a stable input layout and the runtime a fixed memory footprint.

```
action = policy(observation, ActiveContext)
```

The complete knowledge bank is not an input to every forward pass.

This separation preserves exact structure. The relation “the left knee connects the left thigh and shin” is explicit topology, not nearest-neighbor similarity. A learned state such as “the current motion resembles the start of a backward fall” uses a compact tensor representation. A temporal fact such as “the right foot began slipping 60 ms ago” lives in per-environment episodic memory. Each form is represented according to what it means and how the control loop consumes it.

3. Method

This paper uses a structured architectural meta-analysis of primary papers, official technical documentation, and NumiLab documentation at commit [d769f551cfad70082f376d31870efe34c6b5acaf](#). The synthesis focuses on accelerator placement, memory access, policy execution, explicit neural memory, and Apple GPU resource management. Published simulator frame rates remain separate because robots, solvers, timesteps, sensors, precision, policies, and learning workloads differ across systems. The analysis therefore compares architectural mechanisms directly and reports matched NumiLab measurements in their original configurations.

The evidence falls into three groups. GPU robotics systems describe accelerator-resident physics and policy execution [1–7]. Memory research describes explicit bounded memory and compact adaptation state [18–21]. Apple platform documentation describes storage modes, resource binding, residency, GPU scheduling, machine-learning encoding, and profiling [8–17]. NumiLab documentation supplies the current persistent Metal execution architecture and within-system performance measurements [25,26].

Published simulator frame rates remain attached to their original systems because each number encodes a different robot, solver, timestep, sensor load, precision level, policy, and learner. The synthesis therefore extracts repeated architectural patterns across systems and uses matched NumiLab comparisons only where the underlying workload is held constant [23,24].

Evidence class	Contribution	Architectural implication
External simulator architecture	Shows where physics, state, observations, and policy execution live	Establishes accelerator-resident execution patterns
Apple platform specification	Defines Metal and Apple Silicon memory and execution mechanisms	Maps the architecture onto Apple-native primitives
NumiLab matched measurements	Shows the effect of concrete changes inside the same runtime	Identifies active-work elimination and on-chip storage pressure as material costs

Evidence class	Contribution	Architectural implication
Runtime architecture	Combines the recurring mechanisms into one NumiLab execution design	Defines memory placement, execution order, and agent knowledge flow

Table 3. Evidence is separated by what it can legitimately establish.

4. Results of the evidence synthesis

4.1 Keep the control loop on the accelerator

The strongest repeated finding is that large-scale robot learning benefits when physics state moves directly into observation, reward, and policy calculations without a CPU round trip. Isaac Gym exposed physics buffers as GPU tensors and reported large training speedups compared with CPU-simulation pipelines [2]. Brax compiled simulation and learning operations for the same accelerator [3]. Isaac Lab extends the same principle to richer sensing and multi-modal workloads, while explicitly noting that CPU orchestration and launch boundaries remain potential overheads [4].

The control state remains on the GPU and synchronization occurs at deliberate publication boundaries. Physics, event extraction, context selection, policy inference, reward, and rollout publication use separate stages where useful, while intermediate control state stays device-resident between those stages.

4.2 Compile and preallocate what is known in advance

MuJoCo’s native runtime uses data structures preallocated by its model compiler, reducing dynamic allocation and name lookup during stepping [1,6]. GPU simulators similarly rely on fixed or bounded arrays because thousands of environments must execute the same program with predictable memory. NumiLab already compiles robot, scene, task, sensor, policy, and run information into stable tables and fingerprints before execution [25,26]. This supports the same approach for shared knowledge: resolve names, relations, capacities, and layouts before the run; execute only offsets, counts, and IDs during the run.

4.3 Large memory is useful only when access remains bounded

Memory-augmented neural networks show that a neural controller can read and write explicit memory [18]. They also show the main scaling problem: if every step compares against every memory entry, cost grows with memory size. Sparse Access Memory was designed specifically to avoid dense reads and writes as memory grows [19]. Token Turing Machines use a bounded set of memory tokens so that each new observation does not process the complete history; the approach was evaluated for vision-based robot policy learning [20].

Explicit-memory neural systems establish the systems principle that the resident bank is large while the per-step read set remains small [18–21]. NumiLab applies that principle with direct ID selection and bounded semantic operations. Global dense attention over every skill, object, or historical event is replaced by compact candidate selection and a small ActiveContext.

4.4 Separate slow adaptation from fast control

Rapid Motor Adaptation separates a base locomotion policy from an adaptation module that compresses recent history into a compact latent context [21]. NumiLab applies the same control principle at the runtime level: recent experience is reduced to a bounded control context before action selection, keeping the policy input small even as the underlying history grows.

4.5 Five findings that survive differences between systems

#	Finding	Implication for NumiLab
1	Accelerator continuity	Keep physics state, observation construction, context, and policy on the GPU timeline.
2	Compiled structure	Resolve stable semantics and capacities before execution; avoid runtime name lookup and unbounded allocation.
3	Bounded access	Use a small active set rather than scanning all shared knowledge or all history.
4	Representation by semantics	Use tables for exact facts, tensors for learned similarity, and ring buffers for recent events.
5	End-to-end execution	Optimize the complete control path rather than isolated kernels or raw FPS.

Table 4. Repeated architectural findings extracted from the evidence.

5. Apple Silicon and Metal: the relevant mechanisms

5.1 Unified memory is shared capacity, not zero-cost access

Apple Silicon gives the CPU and GPU access to one physical memory pool. This removes the mandatory PCIe transfer of discrete CPU-GPU systems, and MLX exposes the same model directly: an MLX array can move between CPU and GPU execution without an explicit device-copy API [17]. Memory bandwidth, cache locality, write hazards, and synchronization still determine access cost, so Metal resource placement remains a central architectural decision [9–11].

A shared Metal resource serves CPU-GPU exchange: commands, staging uploads, diagnostics, and rollout publication. A private Metal resource serves repeated GPU-side access. On Apple Silicon, private storage is an access policy within unified memory rather than a separate VRAM chip, allowing Metal to optimize the resource for GPU use. NumiLab places immutable knowledge, policy weights, ActiveContext, episode rings, and hot simulation state in private GPU resources, while compact exchange buffers remain shared [11].

Data	Placement	Reason
Commands and small updates	Shared	CPU writes; GPU reads at an explicit boundary
Immutable exact knowledge	Private persistent heap	GPU-only repeated reads; one shared copy across environments

Data	Placement	Reason
Policy weights	Private persistent heap	GPU-only inference between policy revisions
ActiveContext and episode rings	Private dynamic buffers	GPU reads and writes every control step
Diagnostics and rollout publication	Shared ring or staged readback	Compact data crosses to CPU/learner at a controlled boundary
Solver/context scratch	Private transient heap	Lifetime-bounded storage that is reused between stages

Table 5. Apple Silicon resource placement for the NumiLab runtime-resident architecture.

Figure 2. Unified memory gives CPU and GPU access to the same physical capacity. Time-critical GPU state remains separate from exchange buffers and transient scratch.

5.2 Stable resource binding and residency

Argument buffers provide a stable directory of buffers and tensors instead of rebinding many resources for every dispatch. Metal 4 argument tables extend this model on current Metal 4 devices [8,12]. A NumiLab knowledge directory contains bindings for body tables, material features, semantic tensors, ActiveContext records, and episodic rings. The directory stays stable during a run; only versioned data or offsets change at explicit publication boundaries.

Residency sets declare which allocations must be available to a command queue or command buffer [8,13]. They are useful for the persistent world, knowledge, policy, and sensor state because those resources change rarely. Residency does not guarantee that every byte remains in an on-chip cache. It means the resources are prepared and available for GPU execution. The main performance requirement remains a compact, coherent access pattern.

5.3 One GPU timeline, with explicit dependencies

Metal command buffers provide the execution order. Metal 4 adds a machine-learning command encoder that places neural execution on the same GPU timeline as compute work [14]. NumiLab uses one encoded dependency chain: physics produces state; event extraction produces invalidation flags; context selection updates affected environments; policy inference consumes the resulting context; action and reward close the step. The CPU remains outside these intermediate stages.

Kernel fusion follows data reuse and on-chip resource pressure. Contact processing, tactile feature extraction, and event generation share contact data and therefore form a natural fused stage. Large policy inference remains separate when combining it with physics increases register or threadgroup-memory pressure and lowers occupancy. Apple GPU counters expose occupancy, memory traffic, and resource pressure directly [16], giving NumiLab a concrete basis for kernel boundaries.

5.4 Dense tensor work belongs only to dense mathematics

Metal 4 supports inline tensor operations and cooperative tensors, and Apple10-family GPUs and later execute supported tensor operations with per-core neural acceleration [15]. NumiLab uses these operations for dense semantic selectors and compact multilayer perceptrons. Irregular graph

traversal and stable-ID lookup remain ordinary Metal compute. Exact body knowledge stays structured data; dense tensor hardware is reserved for dense tensor mathematics.

6. NumiLab runtime-resident knowledge layer

The runtime-resident layer has two phases: compile-time construction and runtime execution. Compile time resolves human-readable authoring into stable numeric data. Runtime consumes only fixed layouts, offsets, counts, and version numbers. This follows NumiLab’s current CompiledRun model and avoids adding a second dynamic knowledge subsystem beside the simulator [25,26].

6.1 Compile-time construction

1. Resolve the body graph, material table, hazard map, skill bank, and semantic IDs into one canonical namespace.
2. Assign stable integer IDs and compile exact relations into contiguous arrays or compressed adjacency tables.
3. Convert semantic tensors to the runtime precision and pack them into contiguous GPU-ready blocks.
4. Calculate capacities for ActiveContext, candidate lists, and episodic rings.
5. Fingerprint the complete knowledge artifact and record that fingerprint in the compiled run and policy metadata.
6. Upload through shared staging buffers and copy long-lived GPU-only data into private resources.
7. Create the argument table or argument buffer and attach the persistent allocations to the appropriate residency set.

6.2 Runtime state

Each environment holds only changing state. It does not own a copy of the body graph, material table, or skill bank. The minimum additional state is an ActiveContext, a bounded event ring, a dirty mask, and small candidate lists. Environments with the same robot and compiled knowledge artifact share the same immutable knowledge allocation.

```
struct KnowledgeDirectory {
    uint64_t bodyNodeOffset, bodyEdgeOffset, materialOffset;
    uint64_t hazardOffset, skillTensorOffset, fingerprint;
    uint32_t bodyNodeCount, skillCount;
};

struct ActiveContext {
    uint16_t bodyIds[8], hazardIds[4], skillIds[4];
    half features[64]; uint32_t revision;
};

struct EpisodeEvent {
    uint16_t type, bodyId; half severity, deltaTime;
    uint32_t payload;
};
```

ActiveContext, event rings, and candidate lists use fixed-capacity aligned records. Overflow is recorded as typed runtime data, preserving deterministic layout and preventing silent changes to policy input.

6.3 One control step

Figure 3. NumiLab control-step sequence. Only environments whose context changed enter the context-update stage.

1. Advance articulated dynamics and solve contacts using the current action.
2. Produce observations and typed events from accepted state and contact data.
3. Set context-dirty bits when an event or state transition changes the knowledge required for the next policy decision.
4. Compact dirty environment IDs into an active list on the GPU.
5. For those environments only, gather exact knowledge by stable ID and run any bounded semantic operation required by the task.
6. Write a new contiguous ActiveContext and increment its revision.
7. Run policy inference from observation plus ActiveContext.
8. Write the next action, reward, termination state, and selected recent events; continue to the next encoded control step.

6.4 Concrete example: a right-foot slip

Assume the right foot changes from stable contact to tangential slip. Contact processing emits a slip-start event with body ID, direction, severity, and elapsed time. The event marks the context dirty. The selector directly loads the right-foot node, its parent ankle and leg relations, the support-foot state, the fall-protection threshold, and a small set of recovery skill IDs. It writes those features into ActiveContext. The policy calculates an action from proprioception, tactile state, and this context. The full body graph and complete skill bank remain shared and are never copied into the environment or scanned by the policy.

When the next control step introduces no relevant state change, the existing ActiveContext remains in place. Physics and policy continue without rebuilding that environment. Elapsed-time transitions, new goals, policy revisions, knowledge revisions, and resets mark context dirty through the same compiled event path.

6.5 Exact, semantic, and episodic access paths

Access path	Operation	Typical use
Exact lookup	Stable ID → offset → packed record	Body relation, limit, material, hazard
Small candidate selection	Event/state → candidate IDs → fixed-size score operation	Choose among recovery or locomotion priors
Semantic tensor operation	Compact context vector × bounded skill matrix	Only when learned similarity is necessary
Episodic read	Typed recent-event index or fixed recency window	Slip duration, previous impact, failed recovery attempt

Table 6. Each control decision uses the representation matched to the meaning of the knowledge.

7. Quantitative analysis

7.1 Memory scaling

Let N be the number of environments, K the size of shared knowledge, C the ActiveContext size per environment, and E the episodic-memory size per environment. The defined resident design uses one shared knowledge copy:

$$M_{\text{resident}} = K + N(C + E)$$

A design that duplicated the complete knowledge bank per environment would use:

$$M_{\text{duplicated}} = N(K + C + E)$$

The avoided duplication is $(N - 1)K$. The equation captures the central memory rule: shared knowledge is stored once, while each environment carries only references, `ActiveContext`, and its own episodic state.

For $N = 12,288$, $K = 64$ MiB, $C = 512$ bytes, and $E = 4$ KiB, the resident architecture uses about 118 MiB: 64 MiB of shared knowledge, 6 MiB of `ActiveContext`, and 48 MiB of episodic memory. Duplicating the 64 MiB bank per environment requires 768 GiB before physics, sensors, policy state, or rollout storage.

7.2 Bandwidth scaling

The same distinction applies to memory traffic. At 12,288 environments and 50 control steps per second, reading a 512-byte `ActiveContext` once per policy step requires about 300 MiB/s. Reconstructing that context from a 4 KiB working set for every environment and every step requires about 2.34 GiB/s. At a 5% context-update rate, selector traffic falls to about 120 MiB/s. Event-gated context updates therefore convert context construction from a dense per-step operation into sparse state-dependent work.

Event gating leaves the policy’s normal `ActiveContext` read unchanged. It removes repeated reconstruction from the larger knowledge bank. Context size, event rate, candidate-set size, cache locality, and policy width therefore become direct determinants of runtime memory traffic.

7.3 What NumiLab’s current measurements already show

NumiLab’s repository documents a persistent Metal execution path that reached 10,063 environment control-steps/s at 12,288 environments on a 24 GiB M4 Pro system, and 10,265 at 16,384 environments, where retained memory reached 14.1 GB before visual and learner state [26]. A $12,288 \times 24$ visual training update completed 294,912 transitions at 5,532 end-to-end environment steps/s with zero GPU failures and KL 0.00204 [26]. These measurements define the operating scale at which resident knowledge, context memory, and selector traffic must fit.

Matched NumiLab change	Reported comparison	Relative change	Architectural result
Mask non-timeout environments before bootstrap critic multiply	5,692 → 5,801 env steps/s	+1.91% throughput; −1.96% GPU time	Skipping inactive work raises end-to-end efficiency.
Compact ABA storage and reuse dead scratch	5,546 → 5,865 env steps/s	+5.75% throughput; −5.55% GPU time; −22.56% threadgroup storage	On-chip storage pressure and occupancy boundaries materially affect the full training path.

Table 7. Matched NumiLab changes showing the cost of inactive work and on-chip storage pressure.

The first comparison demonstrates the value of skipping inactive work. The second demonstrates the effect of threadgroup storage pressure and scratch lifetime on end-to-end throughput.

Together they support the same runtime strategy used by the knowledge layer: compact active sets, bounded working state, and minimal unnecessary memory traffic.

8. NumiLab runtime architecture

The runtime is organized in layers. Exact shared knowledge forms the base representation. ActiveContext provides the policy-facing working set. Event-gated updates rebuild contexts only when state changes matter. Episodic rings preserve recent typed events. Semantic tensors add learned similarity over bounded candidate sets. Metal 4 tensor and machine-learning paths accelerate dense neural operations while the exact knowledge path remains ordinary Metal compute.

Layer	Runtime implementation	Role in execution
1. Exact shared knowledge	Compile body, material, hazard, and task tables into one private resident allocation.	One shared source of exact runtime knowledge for every environment.
2. ActiveContext	Add a fixed per-environment context populated through direct indexed gathers.	Feeds only relevant knowledge into policy inference.
3. Event-gated updates	Use dirty masks and GPU compaction to rebuild only contexts affected by state changes.	Turns context construction into sparse GPU work.
4. Episodic rings	Store recent typed events in fixed per-environment rings.	Preserves short temporal history without scanning full trajectories.
5. Semantic selector	Apply a bounded MLP or tensor similarity operation to compact candidate sets.	Adds learned semantic selection without global search.
6. Metal 4 tensor path	Execute dense neural operations through Metal machine-learning encoding or TensorOps on supported Apple GPUs.	Keeps neural execution on the same GPU timeline as physics and knowledge activation.

Table 8. Runtime layers of the NumiLab resident-knowledge architecture.

8.1 Runtime components

- KnowledgeCompiler: resolves authored knowledge, assigns stable IDs, chooses layouts, computes fixed capacities, and produces one fingerprinted runtime artifact.
- KnowledgeResidentSet: owns persistent Metal allocations, resource bindings, and the active knowledge revision for one compiled artifact.
- ContextSelector: consumes environment state, events, and candidate IDs and writes ActiveContext for the compact set of environments whose context changed.
- EpisodeMemory: stores recent typed events in a fixed per-environment ring and resets with the environment state.

- PolicyInputContract: defines observation width, ActiveContext layout, knowledge fingerprint, and policy revision as one compiled interface.
- KnowledgeTrace: records compact ID and event provenance for inspection outside the control path, without introducing text generation or retrieval into runtime execution.

8.2 Data layout

- Exact data uses stable integer IDs and contiguous arrays. Metal kernels perform direct indexed access with no runtime strings or hash-table lookup.
- Parallel fields use structure-of-arrays or compact array-of-structures-of-arrays blocks so adjacent GPU lanes read coherent memory.
- ActiveContext is contiguous, aligned, and written directly in policy-input order, eliminating a separate transpose or packing pass.
- Numerically sensitive physics remains FP32. Knowledge features and neural operations use lower precision where the representation does not require full FP32 range.
- Transient heap regions are reused across non-overlapping execution stages, reducing retained scratch memory without adding intermediate copies.
- Runtime buffers use compiled fixed capacities and typed overflow records, preserving deterministic memory layout and keeping dynamic allocation out of the control loop.

9. Conclusion

The synthesis defines a precise NumiLab architecture. Shared robot knowledge is compiled once, stored once, and accessed by stable IDs. Each environment owns only changing state, a bounded recent-event record, and a small ActiveContext. Meaningful events mark that context for rebuilding, and the GPU updates only the affected environments before policy inference. Exact relations remain exact data, learned similarity remains a bounded tensor operation, and recent history remains bounded per-agent memory.

Apple Silicon fits this architecture because CPU and GPU share one memory capacity while Metal exposes explicit storage modes, heaps, argument tables, residency, indirect execution, command-buffer scheduling, and machine-learning operations. NumiLab uses shared resources for exchange boundaries and private persistent resources for the hot path. The result is a control runtime in which physics, event extraction, knowledge activation, policy inference, action generation, reward, and rollout capture remain on one GPU execution timeline.

The architecture begins with immutable exact knowledge in one private resident allocation and a fixed ActiveContext, then extends naturally to event-gated context updates, episodic rings, semantic selectors, and M5-class tensor acceleration. This creates a single runtime memory system for physical state, innate knowledge, recent experience, and neural control.

References

- [1] Todorov E, Erez T, Tassa Y. MuJoCo: A physics engine for model-based control. IEEE/RSJ IROS. 2012:5026–5033. doi:10.1109/IROS.2012.6386109.
- [2] Makoviyshuk V, Wawrzyniak L, Guo Y, et al. Isaac Gym: High Performance GPU-Based Physics Simulation for Robot Learning. NeurIPS Datasets and Benchmarks. 2021. arXiv:2108.10470.
- [3] Freeman CD, Frey E, Raichuk A, et al. Brax: A Differentiable Physics Engine for Large Scale Rigid Body Simulation. 2021. arXiv:2106.13281.
- [4] Mittal M, Guo K, State G, et al. Isaac Lab: A GPU-Accelerated Simulation Framework for Multi-Modal Robot Learning. 2025. arXiv:2511.04831.
- [5] Zhang H, Yuan S, Yan W, et al. GPUSimBench: Towards Scalable and Reliable GPU-Accelerated Simulators in Embodied AI. 2026. arXiv:2607.13059.
- [6] Google DeepMind. MuJoCo official repository and documentation. Accessed 8 August 2026. <https://github.com/google-deepmind/mujoco>
- [7] Google DeepMind. MuJoCo Playground: GPU-accelerated robot learning and sim-to-real environments. Accessed 8 August 2026. https://github.com/google-deepmind/mujoco_playground

- [8] Apple. Discover Metal 4. WWDC25 session 205. 2025. <https://developer.apple.com/videos/play/wwdc2025/205/>
- [9] Apple. Resource fundamentals. Apple Developer Documentation. Accessed 8 August 2026. <https://developer.apple.com/documentation/metal/resource-fundamentals>
- [10] Apple. MTLStorageMode.shared. Apple Developer Documentation. Accessed 8 August 2026. <https://developer.apple.com/documentation/metal/mtlstoragemode/shared>
- [11] Apple. Copying data to a private resource. Apple Developer Documentation. Accessed 8 August 2026. <https://developer.apple.com/documentation/metal/copying-data-to-a-private-resource>
- [12] Apple. Improving CPU performance by using argument buffers; MTL4ArgumentTable. Apple Developer Documentation. Accessed 8 August 2026. <https://developer.apple.com/documentation/metal/improving-cpu-performance-by-using-argument-buffers> ; <https://developer.apple.com/documentation/metal/mtl4argumenttable>
- [13] Apple. Simplifying GPU resource management with residency sets; MTLResidencySet. Apple Developer Documentation. Accessed 8 August 2026. <https://developer.apple.com/documentation/metal/simplifying-gpu-resource-management-with-residency-sets> ; <https://developer.apple.com/documentation/metal/mtlresidencyset>
- [14] Apple. MTL4MachineLearningCommandEncoder. Apple Developer Documentation. Accessed 8 August 2026. <https://developer.apple.com/documentation/metal/mtl4machinelearningcommandencoder>
- [15] Apple. Running inline ML operations in a shader with Metal 4. Apple Developer sample code. Accessed 8 August 2026. <https://developer.apple.com/documentation/metal/running-inline-ml-operations-in-a-shader-with-metal-4>
- [16] Apple. Finding your Metal app's GPU occupancy. Apple Developer Documentation. Accessed 8 August 2026. <https://developer.apple.com/documentation/xcode/finding-your-metal-apps-gpu-occupancy>
- [17] Hannun A, Digani J, Katharopoulos A, Collobert R. MLX: Efficient and flexible machine learning on Apple silicon. Software. 2023–current. <https://github.com/ml-explore/mlx>
- [18] Graves A, Wayne G, Reynolds M, et al. Hybrid computing using a neural network with dynamic external memory. Nature. 2016;538:471–476. doi:10.1038/nature20101.
- [19] Rae JW, Hunt JJ, Harley T, et al. Scaling Memory-Augmented Neural Networks with Sparse Reads and Writes. 2016. arXiv:1610.09027.
- [20] Ryoo MS, Gopalakrishnan K, Kahatapitiya K, et al. Token Turing Machines. 2022. arXiv:2211.09119.
- [21] Kumar A, Fu Z, Pathak D, Malik J. RMA: Rapid Motor Adaptation for Legged Robots. Robotics: Science and Systems XVII. 2021. doi:10.15607/RSS.2021.XVII.011.
- [22] Lewis P, Perez E, Piktus A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS. 2020. arXiv:2005.11401.
- [23] Campbell M, McKenzie JE, Sowden A, et al. Synthesis without meta-analysis (SWiM) in systematic reviews: reporting guideline. BMJ. 2020;368:l6890.
- [24] Page MJ, McKenzie JE, Bossuyt PM, et al. The PRISMA 2020 statement. BMJ. 2021;372:n71.
- [25] Thabit N. NumiLab README: An Apple-native physics and robotics laboratory. Commit d769f551cfad70082f376d31870efe34c6b5acaf. 2026. <https://github.com/Numi2/numi-lab/blob/d769f551cfad70082f376d31870efe34c6b5acaf/README.md>
- [26] Thabit N. Persistent Metal world graph (METAL_WORLD.md). NumiLab, commit d769f551cfad70082f376d31870efe34c6b5acaf. 2026. https://github.com/Numi2/numi-lab/blob/d769f551cfad70082f376d31870efe34c6b5acaf/docs/METAL_WORLD.md